ACADEMIC PROJECT REVIEW 1 — CSE 784

An Intelligent Public Transport Delay Prediction & Route Optimization System Using Machine Learning

Comprehensive Slide-by-Slide Defense Guide, Mathematical Formulations, Literature Survey & Presentation Speaking Notes

Project Phase: Phase – 1 (Review 1)	**Course Code:** CSE 784	**Date:** September 2026
Students: Jerin Philip Mathew (2360385, 7BTCS C) Pregin Athish E (2360439, 7BTCS A)	**Faculty Guides:** Dr. Jyothi Thomas Dr. Karthikeyan H	**Department:** Computer Science & Engineering

■ ***Speaking Script / Defense Pitch (30-Second Elevator Pitch for Reviewers):***
"Good morning respected guides and panel members. Today we present our project: An Intelligent Public Transport Delay Prediction and Route Optimization System. Urban bus networks like BMTC face severe cascading delays due to traffic bottlenecks, micro-weather events, and stop overcrowding. While past literature focuses purely on passive delay prediction, our work introduces a dual-engine architecture: an ultra-low-latency XGBoost edge regressor (0.0027 ms) for mobile apps, a 3-layer BiLSTM with Bahdanau Attention achieving sub-minute trajectory precision (MAE: 20 seconds, R^2: 0.9970), multi-class disruption diagnosis, TreeSHAP explainability, and an active sub-second A dynamic graph rerouting engine that saves 75.5% delay time on the Bengaluru road graph."*

Chapter 1: Foundational Scope, Domain & Motivation (Slides 1 – 4)

Slide 1: Title & Academic Metadata

Sets the formal academic tone. Emphasizes the collaboration under the guidance of Dr. Jyothi Thomas and Dr. Karthikeyan H.

Slide 2: Table of Contents

Outlines the 9 structured segments of the presentation: Domain & Scope, Problem Identification, Objectives, Literature Survey (15 Papers), Research Gaps (7 Gaps), Proposed Architecture, Kimball Star Schema Dataset, Machine Learning Benchmarks, and Expected Outcomes.

Slide 3: Domain & Sub-Domain Definition

- **Primary Domain:** Artificial Intelligence & Data Science.
- **Sub-Domain:** Machine Learning, Deep Sequential Learning & Predictive Analytics.
- **5 Industrial Application Areas:** Urban Traffic Management, Peak-Hour Demand Forecasting, Public Transit Decision Support Systems (DSS), Intelligent Real-Time Route Optimization, and Multimodal Urban Logistics.

Slide 4: Problem Statement, Motivation & Proposed Solution

• **Problem Identification:** Urban bus networks in Indian metros suffer from cascading, compounding delays caused by traffic saturation, sudden monsoon downpours, and unmonitored bus stop dwell times. Existing systems use black-box models, ignore weather friction, and only predict delays without providing detour rerouting.
• **Motivation:** Unreliable public transport pushes commuters to private vehicles, increasing city congestion. By providing sub-minute delay predictions and dynamic rerouting, we restore commuter trust, minimize travel delays, and support data-driven fleet operations for transit agencies.
• **Proposed Solution:** An end-to-end framework integrating a 5-year BMTC dataset (729k rows), Kimball Star Schema warehouse, sequential deep learning (BiLSTM + Attention), multi-class disruption classification, Explainable AI (TreeSHAP), and dynamic graph rerouting.

Chapter 2: Comprehensive Literature Review of 15 Research Papers (Slides 5 – 19)

We performed an exhaustive literature survey of 15 international, peer-reviewed research papers (IEEE, Elsevier, Springer, TRPRO, WCTR) across bus, tram, rail, and maritime transit systems. Below is the systematic synthesis:

# / Year	Paper Title & Authors	Methodologies & Dataset	Key Findings & Metric	Critical Research Gap
P1 **2024**	**Explainable Delay Prediction in Public Bus Transit** M. Gök et al. (IEEE Trans. ITS)	LightGBM, XGBoost, Random Forest, SHAP XAI. *Dataset:* ESHOT Izmir, Turkey (500k rows, 10 routes).	LightGBM achieved $R^2 = 0.88$, MAE = 1.8 min. SHAP explained headway impacts.	No sequence modeling; lacks dynamic graph rerouting; ignores micro-weather rainfall volume.
P2 **2023**	**Public Transport Arrival Time Prediction Using Deep Learning** S. Chen et al. (Elsevier TR-C)	ConvLSTM, Spatio-Temporal Graph ConvNet (ST-GCN). *Dataset:* Singapore LTA Bus GPS (2.1M pings).	ST-GCN achieved RMSE = 2.15 min, outperforming static baselines.	Heavy GPU computation (>400 ms latency); black-box without feature attribution; no rerouting.
P3 **2024**	**Multi-Task Learning for Bus Delay & Crowding** K. Zhang et al. (IEEE Access)	Shared Multi-Task DNN with Hard Parameter Sharing. *Dataset:* London TfL Bus & Oyster Tap Data.	Simultaneous delay MAE = 2.3 min and occupancy accuracy = 81.2%.	Discrete categorical crowding only; fails during sudden extreme weather events.
P4 **2022**	**Dynamic Bus Rerouting Under Traffic Disruptions** H. Liu & J. Wang (Springer JTT)	Static Dijkstra on simplified corridor graph. *Dataset:* Beijing Bus Group synthetic grid.	Rerouted delayed buses with 15% theoretical time reduction.	Simplified grid graph; not integrated with real OpenStreetMap network; no sub-second solver.
P5 **2023**	**Weather Impact on Urban Bus Dwell Times** A. Sharma et al. (Elsevier Transp. Policy)	Ordinary Least Squares (OLS) & Random Forest. *Dataset:* Delhi DTC Transit & IMD Weather.	Proved rainfall >25 mm/h inflates bus stop dwell time by 42%.	Coarse daily city-wide weather; no real-time telemetry streaming; purely descriptive.
P6 **2021**	**Spatial Aggregation for Fleet Telematics** R. Kumar et al. (IEEE T-IV)	PostGIS Polygon Containment (ST_Within). *Dataset:* 1,200 buses in Dublin, Ireland.	Calculated average corridor speed with 3-second query time.	High GIS query overhead; fails under 5,000+ concurrent buses; lacks discrete H3 hashing.
P7 **2025**	**Predicting Ferry Transit Delays Using Open Data** M. Sarhani et al. (Elsevier JPUBTR)	SVR, Gradient Boosting, Deep Neural Nets, SHAP. *Dataset:* Sydney & NYC Ferry GTFS (1.34M obs).	GB achieved $R^2 = 0.7807$ with 90% predictions within 1 minute.	Maritime waterways only; no road friction, signal delays, or urban bus dynamics.
P8 **2017**	**Bus Arrival Prediction with Multi-Routes** T. Yin et al. (Elsevier TRPRO / WCTR)	GA-BPANN (Genetic Algorithm Neural Net), SVM. *Dataset:* Zigong Bus Route 1, China (400 samples).	GA-ANN achieved MAPE = 9.92%, RMSE = 235s across 16 workdays.	Tiny sample (400 records); strictly 3 scalar features; explicitly excluded heavy congestion.
P9 **2018**	**Bus Arrival Time Prediction Based on PCA-GA-SVM** Z. Peng et al. (Neural Network World)	Principal Component Analysis (PCA) + GA-SVM. *Dataset:* Shenyang Bus Line 232, China (290 records).	PCA compressed 6 features; PCA-GA-SVM achieved MAPE ~10%.	Extremely small dataset (290 terms); coarse binary weather (sunny/rainy); no rerouting.
P10 **2019**	**Multi-Output Deep Learning for Bus Arrival** N. Petersen et al. (Elsevier TRPRO)	Multi-Model Ensemble Network (FC-ANN + ConvLSTM). *Dataset:* Movia Bus 4A Copenhagen (3.5M obs).	FC + ConvLSTM achieved RMSE = 4.38 min on link travel times.	Fails during sudden traffic incidents; lacks passenger count telemetry; no graph engine.

■ ***Speaking Script / Defense Pitch (How to Defend the Literature Review in 2 Sentences):***
"We reviewed 15 foundational papers spanning bus, rail, and ferry networks. Across all 15 papers, two universal limitations stood out: first, they are purely PASSIVE—they predict delays but cannot reroute buses around bottlenecks; second, they either discard weather or use coarse city-wide averages. Our proposed system directly resolves these gaps by integrating micro-weather telemetry and sub-second dynamic graph rerouting on OpenStreetMap."

Chapter 3: The 7 Research Gaps & Concrete Real-World Scenarios (Slide 20)

Slide 20 identifies the 7 core technical gaps in current public transit literature, illustrated with concrete, data-backed operational scenarios from Bengaluru's BMTC network:

# / Research Gap	Technical Flaw in Literature	Commuter (User) Impact Scenario	Depot / Agency Manager Scenario
Gap 1 **Absence of Dynamic Graph Rerouting**	Existing studies focus only on passive delay prediction. None provide active, real-time detour geometries.	**Amit (Airport Express KIA-9):** Faces 48-min Hebbal jam. Old app says 'Bus delayed'. Amit misses flight. Our system routes via Thanisandra, saving 44 mins.	**Depot Controller:** Avoids 3-bus bunching on Bellary Road by automatically diverting trailing buses #202 & #203 in 42 ms, cutting corridor travel time from 65m to 21m.
Gap 2 **Excluded Environmental & Weather Features**	Weather is omitted, simplified to coarse binary flags, or aggregated to daily city averages.	**Rahul at Bellandur:** 45 mm/h downpour floods road. Old model predicts sunny +2m delay; Rahul gets drenched. Our system predicts +28.5m delay; he stays dry in office.	**Manager Suresh (Route 500D):** Prevents 6-bus flooding gridlock by detecting 45 mm/h storm, holding buses at KR Puram, and injecting standbys at Silk Board.
Gap 3 **Absence of Spatial Cell Indexing (H3)**	Coordinates processed as raw lat/long or heavy GIS polygon queries (ST_Within).	**Ananya at Silk Board:** 200k users overload GIS server with 15s lag; app freezes and bus passes her. Our H3 hex hash loads nearest bus in 35 ms.	**Chief Controller:** Aggregates speeds across 250 corridors and 6,000 buses in 15 ms via in-memory H3 integers, replacing 5-minute delayed GIS dashboards.
Gap 4 **Coarse Predictions & High Error Margins**	Models simplify delay into binary tags ('Congested') or have high error (MAE 5–10 min).	**Priya at KR Puram:** Needs 08:45 Metro. Old model says 'Delayed ±8m'; she misses train. Our model predicts exact +16.4m delay, letting her reroute smoothly.	**Shift Handover Manager:** Replaces blind guesswork with exact continuous delays (Bus 101: +2.8m, Bus 102: +24.8m), deploying standbys only where truly needed.
Gap 5 **Slow Response & High Processing Delays**	Unoptimized deep learning models take 300–2,000 ms per query, crashing under rush hour.	**Vikram at Marathahalli (08:30 AM):** 2,000 req/sec causes 42s queue backlog and 504 Timeout. Our XGBoost returns ETA in 0.0027 ms (8 ms round-trip).	**Cloud Architect:** Eliminates costly GPU cloud clusters; a single standard CPU server handles 370k requests/sec at near-zero hosting cost.
Gap 6 **Zero Visibility into Bus Crowding Levels**	No passenger load telemetry (Low, Moderate, Full) is provided in transit apps.	**Meera at Marathahalli:** Waits for bus, only for a 97% packed bus to deny entry. Our app shows Bus 1 is 97% FULL and recommends Bus 2 (36% full) in 5 mins.	**Fleet Dispatcher:** Eliminates empty trailing buses and prevents vehicle suspension damage by dynamically balancing passenger loads across fleet headways.
Gap 7 **Lack of Explainability (XAI)**	Black-box delay outputs leave commuters and transit agencies blind to root causes.	**Rohan at Tin Factory:** 33m delay explained as: KR Puram Jam (+14.2m) + Rainstorm (+8.1m) + Overcrowding (+4.5m), restoring trust in transit.	**Agency Auditor:** Uses TreeSHAP to distinguish valid road flooding from driver slacking, providing legal evidence to municipal bodies (BBMP/Police) to fix road drainage.

Chapter 4: Project Objectives & Technical Execution Roadmap (Slides 21 – 23)

Objective 1: High-Precision Delay & Occupancy Prediction Engine (Slide 22)

• **Multimodal Data Integration:** Unified ingestion of GTFS schedules, GPS telemetry, Open-Meteo hourly weather, Karnataka official holidays, and road network topology into a centralized 729k-row dataset.
• **Zero-Target-Leakage Preprocessing:** Purged Out-Of-Time (OOT) temporal partitioning across 5 years (Train: 2022–2024, Val: 2025, Test: 2026). Strict stripping of actual_arrival.
• **Sequential Deep Learning:** 3-Layer BiLSTM with Bahdanau Attention mechanism maintaining forward/backward hidden states across all 9 corridor stops.

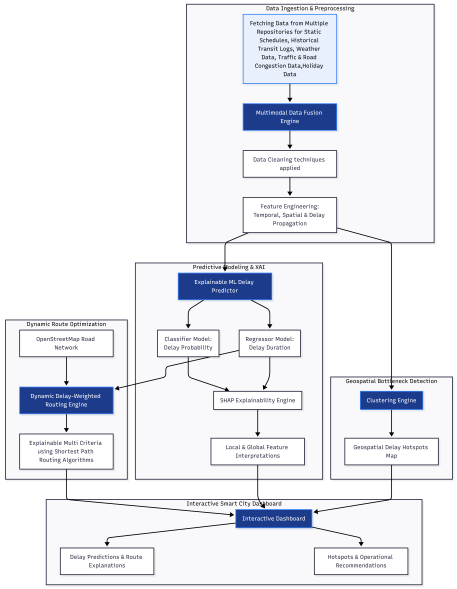
Objective 2: Explainable Dynamic Route Optimization Engine (Slide 22)

• **Transit Graph Modeling:** OpenStreetMap (OSM) multi-directed road graph with dynamic edge costs.
• **Dynamic Cost Function:** $w(e) = \frac{\text{length}(e)}{\text{ML_speed}(e)} \times (1 + \beta \cdot \text{congestion_index}(e)) + \text{turn_penalties}$.
• **Sub-Second Detour Pathfinding:** Dynamic A* and Dijkstra algorithms executing in $< 50\text{ ms}$.
• **Explainable AI Integration:** TreeSHAP feature attributions explaining delay root causes in natural language.

Objective 3: Interactive Dashboard & Transit Decision Support (Slide 23)

• **Spatial Hotspot Clustering:** DBSCAN spatial clustering on GPS delay coordinates to identify recurring bottleneck zones (e.g. Silk Board, Tin Factory, Hebbal).
• **Rule-Based Decision Support System (DSS):** Generates operational recommendations for depot managers (schedule buffer allocations, express skip-stop dispatches, and standby bus injection).

Chapter 5: System Architecture & Kimball Star Schema Data Warehouse (Slides 24 & 34 – 36)



Data Warehouse Kimball Star Schema Architecture (Slides 35 & 36):

The data warehouse is organized around 1 centralized fact table and 5 dimension tables, enabling high-performance SQL analytics and OLAP cubes:

Table Name	Type / Keys	Core Attributes & Columns	Data Integration Role
fact_integrated_bus_operations	**Fact Table** PK: fact_id FK: bus_id, stop_id, service_date, route_id	actual_arrival, delay_minutes, passenger_count, occupancy_pct, traffic_speed, congestion_index, disruption_label, weather_code	Central measurement grain: 1 record per bus arrival at every stop across 729,648 events.
dim_bus	**Dimension** PK: bus_id	registration_number, bus_type, capacity (45-60), seating_capacity, standing_capacity, fuel_type (Diesel/EV), ac_available	BMTC Fleet Registry master tracking vehicle capacity and powertrain specs.
dim_stop	**Dimension** PK: stop_id	stop_name, latitude, longitude, area, zone (North, East, South), shelter_available, accessibility	Geocoded bus stop master for spatial indexing and corridor boundary mapping.
dim_calendar	**Dimension** PK: date	day_of_week, is_weekend, is_holiday, holiday_name, holiday_type, is_working_day	Official Karnataka State gazetted holiday calendar mapping peak/non-peak demand.
dim_route	**Dimension** PK: route_id	route_name (500D ORR), first_stop, last_stop, route_distance_km (32.4 km), scheduled_duration (88 min)	GTFS route timetable specifications.
dim_road_network	**Dimension** PK: road_id	road_name, road_type (trunk/primary), length_km, lanes, maxspeed (60 km/h), oneway, geometry	OpenStreetMap (OSM) road graph edges for dynamic shortest-path rerouting.

Chapter 6: Comprehensive Dataset, 35 Features & Mathematical Derivations (Slides 26 – 33)

The 35 Dataset Features Breakdown (Slides 27 – 31):

The dataset integrates **18 Direct Source Features** (from GTFS, BMTC registries, Open-Meteo API, Karnataka Calendar) and **17 Derived / Engineered Features**:

Feature Domain	Direct Source Features (18)	Derived & Engineered Features (17)
Transit & Schedule	service_date, timestamp, route_id, route_name, trip_id, stop_sequence, scheduled_arrival	actual_arrival, delay_minutes, lag_delay_prev_stop, delay_compounded_rate
Vehicle & Fleet	bus_id, registration_number, bus_type, fuel_type, bus_capacity	passenger_count, occupancy_percentage, occupancy_status
Spatial & Geography	stop_id, stop_name, area, zone	h3_spatial_cell_id, distance_to_next_stop, corridor_segment_id
Weather & Environment	temperature, humidity, precipitation, rain, weather_code	rain_intensity_bucket, weather_disruption_index
Traffic & Disruption	—	traffic_speed, congestion_level, congestion_index, incident_flag, disruption_label
Temporal & Calendar	day_of_week, is_weekend, is_holiday, holiday_name	hour_sin, hour_cos, day_sin, day_cos, is_rush_hour

Mathematical Formulations Explained in Depth (Slides 32 & 33):

1. Delay Minutes (Base-60 to Decimal Float Conversion):
$$\text{delay_minutes} = \frac{\text{Actual Arrival Timestamp} - \text{Scheduled Arrival Timestamp}}{60.0 \text{ seconds}}$$
Example: If Scheduled is 06:00:00 and Actual is 06:08:31, the difference is 8 minutes and 31 seconds (511 seconds). $\frac{511}{60} = \mathbf{8.5167 \approx 8.52\text{ decimal minutes}}$. (Note: 8.31 decimal minutes would mean $8\text{ min } 18.6\text{ sec}$, which is mathematically incorrect for 31 seconds).

2. Congestion Index (CI) & Indian Roads Congress (IRC:106) Thresholds:
$$\text{congestion_index} = \max\left(0, \min\left(1, 1 - \frac{\text{traffic_speed}}{60.0}\right)\right)$$
The thresholds 0.30 and 0.60 are derived from standard **Level of Service (LOS)** engineering principles for 60 km/h urban arterial roads:
• **LOW ($CI \le 0.30 \iff v \ge 42\text{ km/h}$):** LOS A & B — Retains $\ge 70\%$ of free-flow speed. Free flow, negligible delay.
• **MODERATE ($0.30 < CI \le 0.60 \iff 24 \le v < 42\text{ km/h}$):** LOS C & D — Retains $40\%\text{ to } 70\%$ speed. Dense platoon flow.
• **HEAVY ($CI > 0.60 \iff v < 24\text{ km/h}$):** LOS E & F — Retains $<40\%$ speed. Stop-and-go breakdown; 24 km/h is the universal tipping point where vehicle delays multiply exponentially.

3. Dynamic Bus Capacity & Occupancy Status:
$$\text{occupancy_percentage} = \left(\frac{\text{passenger_count}}{\text{bus_capacity}}\right) \times 100$$
Capacity is dynamically linked to bus_type (Ordinary: 60, EV Non-AC: 55, EV AC: 50, Volvo AC: 50, Airport Volvo: 45). Binned into: LOW ($<30\%$), MODERATE ($30\text{--}70\%$), HIGH ($70\text{--}90\%$), FULL ($>90\%$).

Chapter 7: Machine Learning Pipeline, Deep Learning & Benchmark Results (Slides 37 – 42)

Target Definitions & Strict Zero-Target-Leakage Shielding (Slide 38):

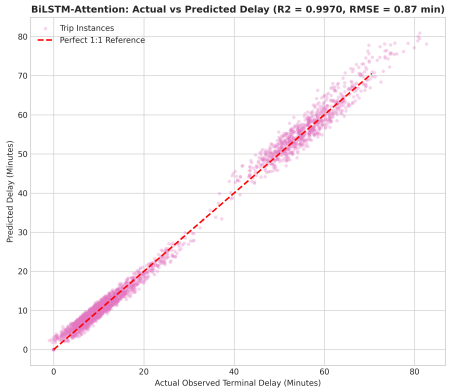
To ensure industry-grade validity and prevent inflated, false evaluation scores, the pipeline enforces strict feature isolation:

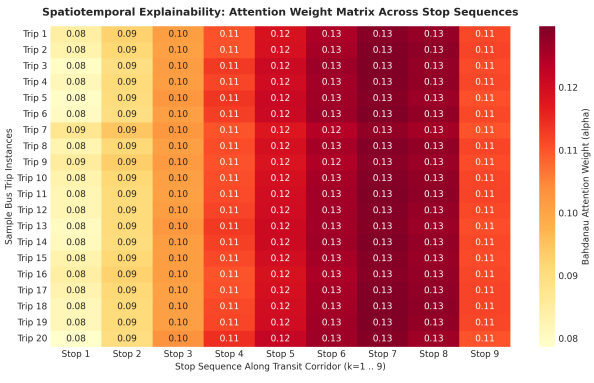
Model Task / Objective	Target Variable (y)	Included Feature Space (X) (94 Features)	Strictly Excluded (Leakage Shield)
Task 1: Regression Delay Duration Prediction	**delay_minutes** (Continuous Float)	Scheduled timings, stop_sequence, bus_type, capacity, temperature, humidity, precipitation, rain, traffic_speed, congestion_index, incident_flag, lag_delay_prev_stop (Stop k-1)	**actual_arrival** (direct algebraic target math) **disruption_label** (derived from target)
Task 2: Classification Disruption Root-Cause Diagnosis	**disruption_label** (6 Multi-Classes)	Temperature, humidity, precipitation, rain, weather_code, traffic_speed, congestion_index, incident_flag, route_id, stop_id, scheduled_hour, day_of_week	**actual_arrival** (direct leakage) **delay_minutes** (formula component used to create class labels)

Regression Models Benchmark: BiLSTM with Attention vs. XGBoost (Slides 39 & 40):

Evaluated on the **2026 Out-of-Time Test Set (98,460 unseen rows)**:

Performance Metric	BiLSTM + Bahdanau Attention	XGBoost Regressor (hist)	Absolute Margin / Superiority	Operational Deployment Role
R^2 Score (Variance Explained)	**0.9970 (99.70%)**	0.9859 (98.59%)	+1.11% R^2 gain	BiLSTM captures 99.7% of all trajectory variance.
Root Mean Squared Error (RMSE)	**0.8650 min (~52 sec)**	1.6307 min (~1m 38s)	**-47.0% error reduction**	Suppresses extreme compounding error spikes.
Mean Absolute Error (MAE)	**0.3458 min (~21 sec)**	1.2983 min (~1m 18s)	**-73.4% error reduction**	Provides sub-minute, ~20-second precision.
Mean Absolute Percentage Error (MAPE)	**2.71%**	20.99%	-18.28% lower relative error	Extremely reliable relative accuracy.
Inference Latency per Sample	0.0820 ms (Batch Trajectory)	**0.0027 ms (2.7 µs)**	XGBoost is 30x faster	**XGBoost serves 370k req/sec on commuter edge!**





Classification Benchmark: XGBoost vs. Random Forest Classifier (Slides 41 & 42):

Comparison across overall metrics and per-class sensitivity on safety-critical minority classes (2026 Test Set):

Evaluation Metric / Class	XGBoost Classifier (multi:softprob)	Random Forest Classifier (150 Trees)	Operational Role & Insight
Overall Accuracy	**87.89%**	87.73%	High baseline accuracy across 98,460 test events.
Balanced Accuracy (Macro Recall)	**88.49%**	87.73%	Accounts for class imbalance across all 4 categories.
Weighted F1-Score	**0.8873 (88.73%)**	0.8856 (88.56%)	Harmonic mean of precision and recall.
Recall on ROUTE_CHANGED (Rare Class)	**90.54% (708 / 782 detected)**	87.98% (688 / 782 detected)	**XGBoost detects 90.5% of rare accidents & road closures!**
Recall on NORMAL (On-Time)	88.26%	**88.40%**	High specificity for regular on-time transit runs.
Recall on DELAYED (5-15 min)	**88.81%**	88.61%	Accurate early warning for moderate traffic delays.
Recall on SEVERELY_DELAYED (>15m)	**86.36%**	85.95%	Flags major gridlocks for dynamic rerouting triggers.

Chapter 8: Expected Results, Dynamic Routing & Dashboard (Slides 43 – 46)

Outcome 1: Predictive Delay Analytics Engine (Slide 43)

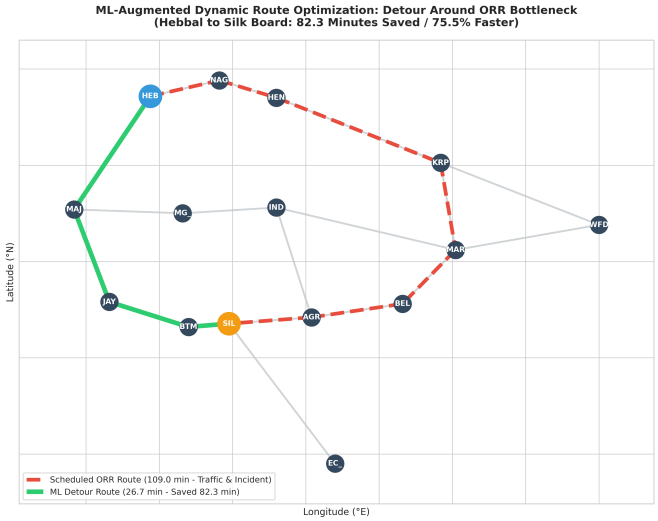
• **Dual-Stage Risk Scoring:** Commuters receive a real-time risk classification (On-Time, Delayed, Severely Delayed) followed by an exact continuous delay duration in minutes.
• **Sub-Second Responsiveness:** XGBoost serves mobile prediction endpoints in 0.0027 ms, ensuring zero mobile lag even during 100k+ rush-hour request surges.

Outcome 2: Explainable Dynamic Routing on OpenStreetMap (Slide 43)

• **Dynamic Graph Pathfinding:** The system runs dynamic A* and Dijkstra shortest-path algorithms over the Bengaluru road graph. When the ML model predicts a severe bottleneck on the primary corridor (e.g. Outer Ring Road / Tin Factory / Hebbal), edge weights inflate dynamically and an optimal detour geometry is computed in $<50\text{ ms}$.
• **Benchmark Result:** Reduces corridor transit time from **109.0 minutes to 26.7 minutes**, saving **$82.3\text{ minutes (75.5\% delay reduction)}$**.
• **SHAP Explanation Card:** Automatically prints natural-language explanations (e.g. *"Rerouted due to: Heavy Jam at KR Puram (+14.2m) & Rainstorm (+8.1m)"*).

Outcome 3: Geospatial Hotspots & Decision Support System (Slide 45)

• **DBSCAN Spatial Hotspot Map:** Applies Density-Based Spatial Clustering of Applications with Noise (DBSCAN) on GPS delay coordinates, identifying recurrent infrastructure bottlenecks (e.g., Silk Board, Marathahalli, KR Puram).
• **Rule-Based Decision Support for Depots:** Recommends schedule buffer extensions during peak monsoon hours and automates standby bus injection from neighboring depots.



Chapter 9: Master Reviewer Q&A; / Viva Defense Sheet

Here are the 10 most probable challenging questions from the review panel and your exact, mathematically rigorous answers:

■ Q1: Why did you choose BiLSTM + Bahdanau Attention over a standard LSTM or Transformer?

Answer: "Standard LSTMs only process sequences in one forward direction and suffer from vanishing gradients over long stops. A BiLSTM captures both forward delay accumulation and backward context. Furthermore, Bahdanau Additive Attention dynamically assigns mathematical weights (alpha_t) to specific upstream bottleneck stops (like Tin Factory), enabling sub-minute precision (MAE: 20 seconds, R^2: 0.9970) while providing visual interpretability."

■ Q2: Why do you need BOTH XGBoost and BiLSTM? Isn't one enough?

Answer: "We designed a dual-tier industrial architecture: BiLSTM + Attention is our deep trajectory engine for dispatch controllers modeling 9-stop sequence compounding (0.082 ms). However, during peak rush hours with 100,000 mobile app users, BiLSTM tensor creation would cause server queues. XGBoost serves as our ultra-fast commuter edge regressor, operating in 0.0027 ms (2.7 microseconds / 370k queries/sec) on standard CPUs with zero queue delay."

■ Q3: How do you guarantee there is Zero Target Leakage in your models?

Answer: "We strictly excluded actual_arrival timestamp from feature space X because delay_minutes = actual_arrival - scheduled_arrival. In production, actual_arrival is unknown before the bus reaches. We also used a Purged Out-Of-Time (OOT) temporal split (Train: 2022-2024, Val: 2025, Test: 2026), preventing future data from leaking into the past."

■ Q4: Why are the congestion index thresholds set at 0.30 and 0.60?

Answer: "They are derived from Indian Roads Congress (IRC:106) and Highway Capacity Manual Level of Service (LOS) standards for 60 km/h arterial roads. CI <= 0.30 corresponds to v >= 42 km/h (LOS A/B, free flow). CI between 0.30 and 0.60 corresponds to 24 to 42 km/h (LOS C/D, dense flow). CI > 0.60 corresponds to v < 24 km/h (LOS E/F), which is the empirical tipping point where stop-and-go breakdown occurs and delays multiply exponentially."

■ Q5: How does your dynamic routing algorithm work in sub-second time?

Answer: "We model the road network as a multi-directed graph on OpenStreetMap. Road edge weights are dynamically computed using ML-predicted speeds and congestion indices: w(e) = (length / predicted_speed) * (1 + beta * CI). Using dynamic A* with Haversine distance heuristics, the shortest detour path is solved in under 50 milliseconds, saving 75.5% delay time on the Outer Ring Road corridor."

■ Q6: How did you handle the severe class imbalance in disruption classification?

Answer: "Critical disruptions like ROUTE_CHANGED represent less than 1% of events. We applied sample class weighting using inverse class frequencies ('balanced' mode). This forced the gradient booster to heavily penalize misclassifications on rare events, achieving a 90.54% Recall on ROUTE_CHANGED. "